

UNIVERSIDAD DE GUADALAJARA

ARQUITECTURA DE SISTEMAS DE INTELIGENCIA ARTIFICIAL

Introducción a Arquitectura de Sistemas de Inteligencia Artificial

Mtro. David Echavarry

16 de abril de 2026

Agenda

Temas

- Objetivo
- Terminología
- Logro de la calidad del sistema
- Procesos del ciclo de vida
- Pruebas del sistema
- Despliegue del sistema
- Operación y monitoreo del sistema
- Análisis del sistema
- Arquitectura de software
- El rol de la arquitectura
- Logro de la calidad
- Recursos
- Calidad del modelo de IA
- Datos
- Tipos de modelos de IA
- Ciclo de vida del desarrollo del modelo
- Asignación de recursos para componentes de IA
- Manejo de la incertidumbre
- Seguridad

Objetivo

Idea clave

- Comprender que un **sistema de IA** no es solamente un modelo entrenado.
- Entender que la calidad del sistema depende de la integración entre:
 - **arquitectura,**
 - **procesos del ciclo de vida,**
 - **modelo y datos.**
- Entender que la arquitectura de un sistema de IA es la combinación de Ingeniería de Software y DevOps en el contexto de IA.
- Entender que en los sistemas de IA existen parte de IA y parte que NO son de IA.

Terminología

Sistema de IA

- Sistema que infiere a partir de la entrada que recibe cómo generar salidas tales como predicciones, contenido, recomendaciones o decisiones que pueden influir en entornos físicos o virtuales.

Arquitectura de Software

- Conjunto de estructuras necesarias para razonar sobre el sistema, las cuales comprenden los elementos de software, las relaciones entre ellos y las propiedades de ambos.

DevOps

- Conjunto de prácticas destinadas a reducir el tiempo entre realizar un cambio en un sistema y poner ese cambio en producción normal, garantizando al mismo tiempo una alta calidad.

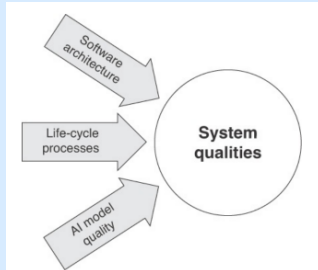
Ingeniería de IA

- Aplicación de los principios y técnicas de la ingeniería de software al diseño, desarrollo y operación de sistemas de inteligencia artificial.

Logro de la calidad del sistema

Calidad de un sistema de IA

- Procesos del ciclo de vida
- Arquitectura de software
- Modelo de IA.



Logro de la calidad del sistema

Atributos de calidad

- Es una propiedad medible o verificable de un sistema que se utiliza para indicar qué tan bien el sistema satisface las necesidades de sus interesados más allá de la función básica del sistema.
- Se hacen para las partes de IA y no-IA por igual.
- Ejemplo: Para la porción de IA sería:
 - precisión
 - robustez
 - equidad
 - restricciones obligatorias
- Ejemplo: Para la porción de no-IA sería:
 - rendimiento
 - seguridad
 - disponibilidad
 - capacidad de modificación

Logro de la calidad del sistema

ODD-data

- ODD-data (out of distribution data) es aquellos datos que no son parte del entrenamiento por definición.
- Siempre habrá ODD-data, existen manera para mitigar esto:
 - Ampliar el conjunto de datos de entrenamiento para incluir la mayor cantidad posible de datos fuera de distribución (OOD)
 - Utilizar algoritmos de entrenamiento más avanzados que puedan generalizar mejor
 - Emplear enfoques externos al modelo para detectar y gestionar los datos fuera de distribución (OOD).

Procesos del ciclo de vida

Idea clave

- Un sistema tiene un ciclo de vida.
- Los componentes del sistema son desarrollados y puestos a prueba de manera aislada y después integrados para hacer el sistema completo.
- Se usan algunas técnicas para crear la parte de IA y la parte de no-IA.
- Ambas partes se integran para construir un artefacto ejecutable. Una vez que se construye el ejecutable, el sistema avanza a través de un pipeline que implica el uso intensivo de herramientas para probar el ejecutable y desplegar el resultado.

Procesos del ciclo de vida

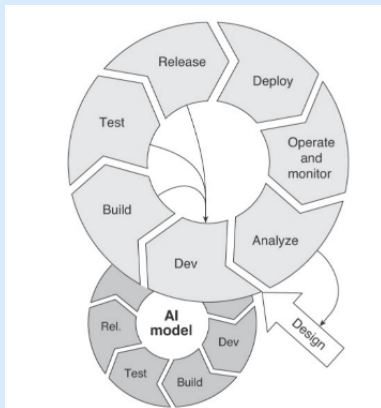


Figura: Ciclo de vida

Procesos del ciclo de vida

Componentes importantes del ciclo de vida

- Desarrollo del modelo: selección, exploración, entrenamiento y ajuste de los modelos de IA.
- Etapa de desarrollo (Dev): realizar actividades normales de desarrollo y crear scripts para otras actividades relacionadas con el sistema de IA.
- Desarrollo del sistema: creación de artefactos ejecutables para todo el sistema.

Pruebas del sistema

Algunas pruebas de sistema importantes

Una vez que el sistema es construido, debe de someterse a pruebas automatizadas. Las pruebas suelen ser:

- Pruebas de regresión
- Pruebas básicas (smoke testing)
- Pruebas de compatibilidad
- Pruebas de integración
- Pruebas funcionales
- Pruebas de usabilidad
- Pruebas de instalación/desinstalación
- Pruebas de calidad

Solo una vez que el sistema pasa estas pruebas, se puede desplegar.

Despliegue del sistema

De despliegue a producción

Implica mover el sistema a ambiente de producción. Esto implica:

- Crear infraestructura (servidores, cloud, contenedores)
- Configurar el sistema (variables, conexiones, APIs)
- Cambiar de versión (old → new)
- Problema clave: actualizar el sistema
- 2 formas para actualizar el sistema:
 - Downtime: apagas todo y actualizas
 - Live Update: actualizas sin apagar el sistema
- Debes diseñar el sistema DESDE EL INICIO para soportarlo.

Operación y monitoreo del sistema

De despliegue a producción

Cuando el sistema ya de despliega, entra en la fase de operación y monitoreo. Durante esta fase, el sistema es monitoreado y medido.

- logs (errores, eventos)
- métricas (CPU, latencia)
- eventos (acciones del sistema)
- input/output del modelo
- El sistema DEBE de generar alertas
- Dos tipos de monitoreo:
 - Sistema externo: recoge datos y generar alertas
 - Parte del sistema: propio sistema tiene lógica de monitoreo

El monitoreo NO es automático, debe diseñarse en la arquitectura. Un sistema sin monitoreo en producción es un sistema que no entiendes.

Análisis del sistema

Análisis del sistema

- Es la etapa donde se interpretan los datos recolectados durante la operación y monitoreo.
- Convierte métricas, logs y eventos en conocimiento útil (insights).
- Permite entender por qué ocurren ciertos comportamientos en el sistema.
- Ayuda a identificar áreas de mejora:
 - Rediseño de arquitectura
 - Optimización de código
 - Reentrenamiento del modelo
 - Mejora del pipeline
- En sistemas de IA, permite detectar problemas como:
 - Degradación del modelo
 - Sesgos
 - Datos fuera de distribución (drift)
- Cierra el ciclo de vida: análisis → mejora → nuevo diseño.
- Idea clave: monitorear dice qué pasa, analizar explica por qué pasa.

Arquitectura de Software

Arquitectura

- Diseñar un sistema de software implica crear su arquitectura.
- La arquitectura define los elementos del sistema y las relaciones entre ellos.
- Su objetivo es cumplir tanto metas funcionales como de calidad.
- La arquitectura funciona como un plano (blueprint) para la construcción del sistema.
- Permite asignar funcionalidades a los distintos componentes.
- La arquitectura contiene los componentes donde se ejecuta el modelo de IA.
- El modelo depende de los datos, pero la arquitectura no depende directamente de ellos.
- La arquitectura está influenciada por:
 - Requisitos de calidad del sistema
 - Tecnologías disponibles (por ejemplo, la nube)
 - Recursos disponibles

El rol de la arquitectura

Partes importantes de la arquitectura

- La arquitectura del sistema actúa como el plano (blueprint) para su construcción.
- La arquitectura debe:
 - Estructurar el sistema en varias partes, como componentes o modelos de IA, y sus relaciones.
 - Especificar la asignación de las distintas partes del sistema a los recursos adecuados.
 - Permitir el monitoreo durante la operación y la generación de alertas ante incidentes.
 - Incorporar una estrategia de despliegue y actualización.
 - Soportar despliegue continuo, permitiendo múltiples versiones activas simultáneamente (version skew).

Logro de calidad

Ideas clave

- La arquitectura determina el logro de los atributos de calidad que no dependen del modelo de IA.
- Para cada atributo de calidad existen técnicas arquitectónicas específicas (tácticas arquitectónicas).
- Ejemplos de tácticas arquitectónicas:
 - Redundancia → para disponibilidad
 - Caché → para eficiencia
 - Soporte a la iniciativa del usuario → para usabilidad
- Para lograr calidad en sistemas de IA no basta con la arquitectura.
- También es necesario aplicar técnicas en la preparación de los datos de entrenamiento.

Recursos

Ideas clave

- Un sistema de IA utiliza diferentes tipos de recursos para su ejecución y entrenamiento.
- Existen tres categorías principales de recursos:
 - Local: computadoras personales como laptops o desktops.
 - Edge: dispositivos como smartphones o IoT con recursos limitados.
 - Cloud: servidores remotos en la nube (pública o privada).
- El sistema suele ser distribuido, ejecutándose en múltiples ubicaciones.
- Las distintas partes del sistema se comunican a través de una red.
- Es necesario comprender redes, protocolos y mecanismos de comunicación.
- El sistema y los modelos de IA se empaquetan en contenedores o máquinas virtuales.
- El empaquetado permite escalabilidad, facilidad de despliegue y manejo de dependencias.
- La nube facilita la ejecución del sistema y el entrenamiento o actualización de modelos.
- La asignación de recursos depende de las técnicas de IA utilizadas.

Calidad del modelo de IA

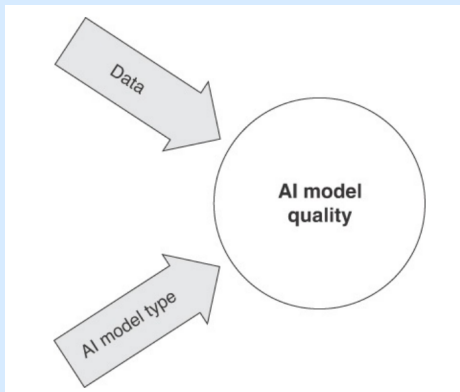


Figura: Calidad modelo IA

Datos

MLOps

- Los sistemas de IA infieren resultados a partir de datos.
- Los datos pueden provenir de múltiples fuentes (bases de datos, APIs, sensores, etc.).
- En la práctica, los datos casi siempre tienen problemas.
- Problemas comunes en los datos:
 - Valores faltantes
 - Outliers
 - Formatos inconsistentes
 - Datos duplicados
 - Baja calidad
- La calidad de los datos afecta directamente la calidad del modelo.
- Es necesario limpiar y organizar los datos antes de usarlos.
- Feature engineering consiste en mejorar las variables de entrada del modelo.
- Ejemplos de feature engineering:
 - Convertir categorías a números
 - Escalar o normalizar datos
- MLOps gestiona todo el ciclo de vida de datos y modelos.
- Idea clave: en IA, los datos son tan importantes como el modelo.

Tipos de modelos de IA

Modelos de IA

- Se utiliza un modelo de IA cuando no existe una solución algorítmica explícita.
- Un modelo de IA se compone de dos elementos principales:
 - Base de conocimiento (knowledge base)
 - Motor de inferencia (inference engine)
- El modelo de IA es ejecutable y puede usarse como función o servicio (ejemplo: APIs de OpenAI).
- Existen tres tipos principales de IA:
 - IA simbólica (basada en reglas IF-THEN)
 - Machine Learning (basado en datos)
 - Foundation Models (FMs)
- Los modelos de Machine Learning incluyen:
 - Clasificación
 - Regresión
 - Clustering
- Los Foundation Models (FMs) son modelos entrenados con grandes volúmenes de datos y de propósito general.
- Ejemplos de FMs: GPT-4, Llama, Mistral.
- Los FMs pueden adaptarse mediante:
 - Prompt engineering
 - Fine-tuning
 - Guardrails
 - Retrieval-Augmented Generation (RAG)
- RAG permite complementar al modelo con información externa relevante en tiempo real.
- Ejemplo de RAG: consultar documentos internos antes de generar una respuesta.
- Idea clave: los sistemas de IA combinan modelos, datos y mecanismos externos para generar resultados.

Ciclo de vida del desarrollo del modelo

Ciclo de vida

- Después del desarrollo del modelo, se construye un artefacto ejecutable (model build).
- El modelo se transforma en un formato desplegable o se habilita el acceso a él (por ejemplo, vía API).
- El objetivo es que el modelo esté listo para integrarse en el sistema.
- El modelo debe ser probado exhaustivamente antes de su uso.
- Las pruebas evalúan:
 - Precisión (accuracy)
 - Riesgos y sesgos (bias)
 - Confiabilidad
- Se utilizan métricas y criterios definidos para validar su desempeño.
- Una vez validado, el modelo pasa a la etapa de liberación (release).
- El release implica aprobar el modelo para su integración en el sistema.
- Idea clave: antes de desplegar un modelo, debe ser construido, probado y validado rigurosamente.

Asignación de recursos para componentes de IA

Idea clave

- La asignación de recursos para la parte de IA depende de la técnica utilizada.
- Los requerimientos de recursos varían según el tipo de modelo.
- Machine Learning (ML):
 - El entrenamiento se realiza en recursos locales o en la nube (ejemplo: laptop o AWS).
 - En algunos casos, puede realizarse en dispositivos edge (on-device learning).
 - El modelo resultante puede ejecutarse en edge si es lo suficientemente pequeño.
 - De lo contrario, se ejecuta en recursos locales o en la nube.
- Foundation Models (FMs):
 - Generalmente se alojan en la nube (ejemplo: APIs de OpenAI, AWS).
 - Se accede a ellos mediante llamadas a APIs o servicios.
 - Algunos modelos pequeños pueden ejecutarse en dispositivos edge (ejemplo: smartphones, smart speakers).
 - Existen técnicas como compresión o distillation para reducir su consumo de recursos.
- Idea clave: la arquitectura y los recursos dependen directamente del tipo de modelo de IA utilizado.

Manejo de la incertidumbre

Idea clave

- Los modelos de IA no simbólica (Machine Learning) son inherentemente probabilísticos.
- Esto implica que siempre existe una probabilidad de error en las salidas del modelo.
- La aceptación de estos errores depende de la tolerancia al riesgo de la organización.
- Es necesario considerar aspectos como:
 - Evaluación de riesgos en IA
 - IA responsable
 - Seguridad en IA
 - Cumplimiento de estándares y regulaciones
- Debido a la incertidumbre en sistemas de IA, se debe realizar un análisis de riesgos.
- Este análisis debe considerar:
 - Valores humanos
 - Seguridad de la IA
- Idea clave: la IA no es determinista, por lo que requiere gestión de riesgo.

Seguridad

Ideas clave

- La seguridad (safety) en sistemas de IA es un tema en constante evolución.
- Inicialmente se enfocaba en la seguridad física y psicológica de las personas.
- Actualmente abarca riesgos mucho más amplios.
- Incluye preocupaciones sobre capacidades peligrosas:
 - Riesgos químicos, biológicos, radiológicos y nucleares (CBRN)
- Considera el potencial de mal uso de la IA:
 - Ciberseguridad
 - Desinformación y manipulación a gran escala
- También incluye:
 - Controlabilidad de los sistemas de IA
 - Posibles riesgos existenciales para la humanidad
- Idea clave: la seguridad en IA va más allá del usuario individual y tiene impacto global.

Bibliografía

Referencia base

- Bass, L., Lu, Q., Weber, I., & Zhu, L. (2025). *Engineering AI Systems: Architecture and DevOps Essentials*. Addison-Wesley.

Complementaria

- Bass, L., Clements, P., & Kazman, R. *Software Architecture in Practice*.
- Géron, A. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*.